Qwen

Конструктор ракет-перехоплювачів для **Juno: New Origins:** створення автономного генератора **XML-**крафтів у **GitHub Codespaces**

Дане дослідження присвячене аналізу та розробці концепції веб-додатку для автоматизованої генерації робочих XML-файлів крафтів для гри *Juno: New Origins* (версія 1.3.205) на платформі Android. Основна мета полягає у створенні інструменту, який не лише спрощує процес проектування складних конструкцій, таких як системи протиракетної оборони класу THAAD, але й гарантує їхню технічну валідність та сумісність з конкретними версіями гри та обмеженнями платформи. Дослідження враховує критичну важливість точних фізичних параметрів, правильного кодування файлів та адаптації процесу завантаження для нових версій Android. У результаті буде представлена комплексна стратегія щодо вибору технологічного стеку, архітектури додатку, механізмів генерації XML та процедур використання, що дозволить створити надійний і корисний рішення для гравців.

Архітектура та Технологічний Стек для **GitHub Codespaces**

Створення ефективного та доступного інструменту для генерації XML-крафтів для *Juno: New Origins* вимагає вибору правильної архітектури та технологічного стеку, який би відповідав поставленим цілям: простоті розгортання, мінімізації вартості та забезпеченню стабільної роботи. На основі аналізу мети дослідження та технічних вимог, найбільш раціональним вибором є використання середовища **GitHub Codespaces** . Це хмарна IDE, яка надає повноцінне програмне забезпечення для розробки прямо в браузері, без потреби в налагодженні локального середовища [10]. Середовище CodeSpaces вже має попередньо встановлені інструменти Python, Flask та Git, що значно

прискорює процес розробки та запуску [11] [23]. Такий підхід дозволяє розробнику та користувачеві отримати доступ до додатку за однією адресою URL, що є ключовою вимогою для створення "веб-сайту".

Центральною частиною бекенд-логіки буде **Python**, який вибрано завдяки своїй простоті, потужності та величезній кількості бібліотек для роботи з XML та побудови веб-серверів. Для створення веб-інтерфейсу та взаємодії з користувачем буде використовуватися легковажний веб-фреймворк **Flask**. Flask є ідеальним вибором для невеликих проектів та API, оскільки він мінімалістичний, гнучкий та легко підходить для задач, подібних до поставленої, де не потрібні складні можливості масштабованого веб-розподілу [9].

Фронтенд-частина додатку, що відповідає за користувацький інтерфейс, буде побудована на стандартних веб-технологіях:

- **HTML:** Для створення структури форми, де користувач введе параметри для генерації крафту (наприклад, тягу двигуна, загальну довжину, кількість ступенів) .
- **CSS:** Для стилізації сторінки та забезпечення її привабливого та зручного вигляду.
- **JavaScript:** Для реалізації інтерактивності, наприклад, для валідації введених даних перед відправкою на сервер, а також для обробки відповіді від сервера та ініціації завантаження згенерованого файлу. JavaScript може бути використаний для створення Blob-об'єктів з XML-даними та їхній швидкого скачування без взаємодії з сервером, що покращує користувацький досвід [27] [29].

Архітектура додатку матиме три основні компоненти: 1. Веб-сервер **(Flask):** Приймає HTTP-запити з форми користувача, передає дані до скрипта генерації та надсилає згенерований XML-файл як відповідь з типом контенту `text/xml` та заголовком `Content-Disposition: attachment; filename="craft.xml"`. Це зробить кнопку "Завантажити" на сторінці дійсною. 2. Скрипт генерації **(Python):** Це ядро системи. Він приймає параметри від Flask, будує дерево XML-елементів за допомогою бібліотеки `xml.etree.ElementTree` та повертає готовий XML-рядок. Важливо, що саме цей компонент вирішує ключову проблему — генерація валідного, а не "вигаданого", XML. Він буде використовувати перевірені шаблони частин, що гарантує фізичну сумісність крафту з грою . 3. Користувацький інтерфейс **(HTML/JS):** Проста сторінка з

однією головною функцією — збором вхідних даних та надсиланням їх на сервер. Це відповідає вимозі про простоту .

Особливу увагу слід приділити проблемі кодування файлу. Для гарантії сумісності з *Juno: New Origins*, особливо на платформі Android, XML-файл повинен бути збережений у кодуванні **UTF-8** без маркера порядку байтів **(BOM)** . Будь-яке відхилення від цього стандарту може призвести до помилки завантаження крафту ("Неприпустимий крафт") . Python-скрипт повинен явно вказувати це кодування при збереженні або надсиланні файлу, щоб уникнути проблем, пов'язаних з некоректною обробкою спеціальних символів [4] [25].

Таким чином, обраний технологічний стек (GitHub Codespaces + Python + Flask + HTML/JS) є оптимальним вибором, оскільки він забезпечує швидке розгортання, низьку складність підтримки, гнучкість у розробці та, що найважливіше, дає змогу створити надійний інструмент, який генерує технічно коректні та сумісні XML-файли.

Дизайн та Реалізація Користувацького Інтерфейсу

Дизайн користувацького інтерфейсу є ключовим фактором успіху додатку, оскільки він визначає простоту та ефективність взаємодії користувача з інструментом. Відповідно до вимоги про простоту , інтерфейс має бути максимально зведеним до необхідного, не перетурбовуючи користувача зайвими опціями. Основним елементом інтерфейсу буде одна центральна форма, яка дозволить користувачеві ввести параметри для генерації крафту.

Форма повинна містити наступні поля введення, кожне з яким має бути чітко позначене:

Поле	Тип	Приклад значення	Опис
Назва крафту	Текстове поле	`THAAD_Interceptor_500km`	Ім'я, яке буде присвоєно згенерованому крафту в грі.
Тип крафту	Випадаюче меню	`THAAD`, `Patriot PaC3`, `Intercept_10Mach`	Предвизначені шаблони з різними характеристиками, що відповідають різним системам ПРО.
Тяга двигуна (Н)	Числове поле	`450000`	Сила тяги двигуна, що впливає на прискорення та TWR.
Об'єм палива (л)	Числове поле	`8000`	Загальний об'єм палива, що визначає ΔV крафту.
Кількість ступенів	Числове поле (1-2)	`2`	Кількість ступенів ракети.
Коефіцієнт опору	Числове поле (0.1-1.0)	`0.2`	Зменшує аеродинамічний опір, що важливо для польотів у верхніх шарах атмосфери.
Коефіцієнт маси	Числове поле (0.5-1.5)	`0.8`	Змінює масу конструкції, впливаючи на загальну інерцію.

Для забезпечення коректності введених даних та покращення користувацького досвіду, інтерфейс повинен мати такі характеристики:

- Валідація на стороні клієнта: JavaScript повинен перевіряти, що числові поля містять коректні значення (наприклад, тяга > 0). Це дозволить вчасно повідомити користувача про помилку, не надсилаючи некоректні дані на сервер.
- Прогрес-бар: Під час генерації складного XML-файлу може виникнути затримка. Показ прогрес-бару або завантажувального індикатора підвищить враження від роботи додатку.
- Чітка кнопка "Згенерувати": Після натискання на цю кнопку скрипт генерації повинен відправити дані з форми на бекенд (сервер на Flask), який обробить запит та надішле файл для скачування. Інтерфейс повинен чітко вказувати, що результатом натискання є початок завантаження файлу.

Окрім основної форми, інтерфейс повинен містити розділ "Допомога" або "Інструкції". Цей розділ є критично важливим, оскільки процес завантаження XML-файлу в гру на Android 16+ є нелінійним і вимагає спеціальних знань . Розділ "Допомога" повинен містити крок-за-крокові інструкції з використання файлового менеджера **MT Manager**. Ці інструкції повинні детально описувати процедуру: 1. Де знаходити папку `Crafts` в файловій системі Android. 2. Як створити "заглушковий" крафт у грі для заміни його вмісту. 3. Як відкрити та

зберегти файл у MT Manager, вказуючи кодування `UTF-8 без BOM`. 4. Як обходити обмеження доступу до файлової системи на Android 11+ [3].

Цей підхід, коли інтерфейс не тільки збирає дані для генерації, але й надає всі необхідні інструкції для подальшого використання створеного файлу, робить додаток справжнім навчальним та практичним інструментом для гравців. Це вирішує одну з найбільших проблем, яку виявив аналіз діалогу з користувачем — складність завантаження файлів на нових версіях Android .

Механізми XML-генерації та Адаптація для Juno: New Origins

Ефективність генератора XML-крафтів для *Juno: New Origins* залежить виключно від точності та сумісності створюваної XML-структури. Прямі спроби AI-генерації часто призводять до некоректних координат, порушених з'єднаннях та, як наслідок, нестабільних або неробочих крафтів, оскільки гра відхиляє файли з помилками в структурі . Тому ключовою вимогою є використання шаблонного підходу на базі Python-скрипта, який точно контролює кожен елемент дерева XML.

Процес генерації повинен базуватися на перевірених блоках, які існують у грі, таких як `CommandPod`, `FuelTank` та `RocketEngine`. Скрипт не "вигадує" параметри, а використовує задані користувачем значення (тяга, об'єм палива) для налаштування цих блоків. Ось приклад функції на Python, яка генерує базову 1-ступеневу ракету:

```
import xml.etree.ElementTree as ET
from xml.dom import minidom

def generate_interceptor(name, thrust, fuel_capacity):
    # Створення кореневого елемента <Craft>
    root = ET.Element("Craft", name=name, version="2")

    # Створення <Assembly>
    assembly = ET.SubElement(root, "Assembly")

    # 1. Командний модуль
```

```
    pod = ET.SubElement(assembly, "Part", partId="0", partType="CommandPod
    ET.SubElement(pod, "position", x="0", y="0", z="0")
    ET.SubElement(pod, "rotation", x="0", y="0", z="0")
    config_pod = ET.SubElement(pod, "config", dragScale="0.8", massScale="
    cp_data = ET.SubElement(pod, "CommandPod")
    ET.SubElement(cp_data, "powerConsumption").text = "10"
    ET.SubElement(cp_data, "gyroPower").text = "50"

    # 2. Бак з паливом
    tank = ET.SubElement(assembly, "Part", partId="1", partType="Fuselage"
    ET.SubElement(tank, "position", x="0", y="-2.0", z="0")
    ET.SubElement(tank, "rotation", x="0", y="0", z="0")
    config_tank = ET.SubElement(tank, "config", dragScale="0.5", massScale
    ft_data = ET.SubElement(tank, "Fuselage")
    ET.SubElement(ft_data, "capacity").text = str(fuel_capacity)
    ET.SubElement(ft_data, "fuelType").text = "0" # 0 = рідке паливо

    # 3. Ракетний двигун
    engine = ET.SubElement(assembly, "Part", partId="2", partType="RocketE
    ET.SubElement(engine, "position", x="0", y="-4.0", z="0")
    ET.SubElement(engine, "rotation", x="0", y="0", z="0")
    config_eng = ET.SubElement(engine, "config", dragScale="0.3")
    re_data = ET.SubElement(engine, "RocketEngine")
    ET.SubElement(re_data, "thrust").text = str(thrust)
    ET.SubElement(re_data, "vacuumEfficiency").text = "310" # Типове значе
    ET.SubElement(re_data, "gimbalRange").text = "5"

    # 4. З'єднання частин
    ET.SubElement(assembly, "Connection", part1="0", part2="1", attachPoin
    ET.SubElement(assembly, "Connection", part1="1", part2="2", attachPoin

    # Форматування XML для читання
    rough_string = ET.tostring(root, encoding='utf-8')
    reparsed = minidom.parseString(rough_string)
    return reparsed.toprettyxml(indent="  ")
```

Цей підхід має кілька переваг:

- Валідність: Гарантується правильна XML-синтаксис та закриття всіх тегів.

- Гнучкість: Легко модифікувати для додавання нових параметрів або ступенів.
- Надійність: Використовуючи перевірені `partType` (`CommandPod`, `Fuselage` тощо), ми забезпечуємо сумісність з грою.

Особливу увагу слід приділити адаптації під версію гри `1.3.205` та платформу Android. Як показує практика, гра може вимагати специфічних полів або значень. Наприклад, для перехоплення цілей 10 Махів потрібні високі показники:

- Тяга: `thrust` в межах 300,000 - 450,000 Н.
- Ефективність: `vacuumEfficiency` понад 380 с.
- Керування: `gimbalRange` не менше 6° для точного наведення на високих швидкостях .

Отже, продвинутий генератор повинен містити вбудовані "профілі" або шаблони для різних типів крафтів (THAAD, Patriot PaC3, інтерцептор 10 Махів). Коли користувач вибирає "THAAD" у випадаючому меню, скрипт не просто використовує введені ним значення, а додає до них параметри, характерні для THAAD: двоступенева конструкція, високий `gimbalRange` для перехоплювача, режим радара для балістичних цілей тощо . Це дозволить генерувати не просто "корабель", а функціональну систему згідно з вимогами.

Нарешті, генератор повинен бути відкритим для розширення. Розробник може додавати нові шаблони, змінювати параметри існуючих або додавати підтримку складніших елементів, таких як програми польоту (`FlightProgram`), які є необхідними для автономного перехоплення балістичних цілей . Такий підхід, що поєднує простоту введення даних з потужністю та гнучкістю Python-генерації, є єдиним надійним способом створення корисного інструменту для гравців *Juno: New Origins*.

Інтеграція з Механікою Перехоплення Балістичних Цілей

Реалізація інструменту, який генерує лише фізично стабільну конструкцію, є недостатньою для досягнення високої мети користувача — створення ефективних систем ПРО класу THAAD . Ключовим елементом, що відрізняє

THAAD від просто потужної ракети, є його здатність до прогнозного перехоплення балістичних та міжконтинентальних балістичних ракет (МКБР). Ці цілі рухаються за інерцією після вимкнення двигуна, тому проста стратегія "переслідування" є неефективною. Генератор повинен інтегрувати логіку, що дозволяє ракеті-перехоплювачу розраховувати точку зустрічі та летіти до неї.

Для цього генерований XML-файл повинен містити не лише фізичну модель ракети, але й програму польоту. Ця програма, записана у вигляді XML-блоку `<FlightProgram>`, виконує логічні операції для наведення ракети на ціль. Аналогічно до того, як реальні системи THAAD використовують радари для трекінгу та обчислень, наш генератор повинен генерувати програму, яка отримує дані про положення та швидкість цілі та обчислює оптимальну траєкторію.

Ось приклад логіки, яку має містити програма польоту для прогнозного перехоплення:

```
<FlightProgram>
  <Program name="THAAD_BallisticIntercept">
    <Variables>
      <Variable name="TargetPos" type="Vector"/>
      <Variable name="TargetVel" type="Vector"/>
      <Variable name="InterceptPoint" type="Vector"/>
      <Variable name="TTI" type="Number"/> <!-- Time To Intercept -->
    </Variables>
    <Instructions>
      <GetTargetData position="TargetPos" velocity="TargetVel"/>

      <!-- Прогнозування точки перехоплення -->
      <PredictIntercept selfSpeed="2900" targetPos="TargetPos" targetVel="
                        output="InterceptPoint" time="TTI"/>

      <If condition="TTI > 0.5"> <!-- Якщо час перехоплення більше 0.5 сек
        <NavigateTo target="InterceptPoint" leadAngle="true" maxG="15"/>
      </If>
      <Else>
        <NavigateTo target="TargetPos" mode="Pursuit" maxG="20"/> <!-- Рез
      </Else>
    </Instructions>
```

```
  </Program>
</FlightProgram>
```

У цьому прикладі використовуються гіпотетичні, але логічно обґрунтовані команди, які могли б існувати в гри:

- ``: Отримує поточне положення (`position`) та вектор швидкості (`velocity`) цілі.
- ``: Це ключова команда. Вона приймає швидкість самої ракети (`selfSpeed` ≈ 2900 м/с для 10 Махів) та дані про ціль, і розраховує майбутню точку (`output`) та час до неї (`time`). Це дозволяє ракеті летіти назустріч цілі, а не доганяти її.
- ``: Команда для наведення. Якщо використовується `leadAngle="true"`, ракета буде випереджати ціль, що критично важливо на гіперзвукових швидкостях .

Крім програми польоту, система THAAD включає в себе радар. Генератор повинен також генерувати або вбудовувати блок радара з налаштуваннями, що дозволяють відфільтрувати балістичні цілі та ігнорувати авіацію. Це можна зробити, встановивши властивості радара:

```
<Radar range="650000" trackMode="Ballistic" filterBallistic="true" ignoreA
```

Такі налаштування, які відображено у коді, дозволять радару сфокусуватися на виявленні балістичних траєкторій .

Таким чином, повноцінний генератор для THAAD повинен мати наступну логіку: 1. Прийняти вибір користувача: "Тип крафту" -> "THAAD". 2. Згенерувати фізичну модель: Двоступеневу ракету з високим `gimbalRange` (до 15) та `gyroPower` (до 150) для перехоплювача, що дозволить йому точно маневрувати в космосі . 3. Згенерувати програму польоту: Вбудовувати XML-блок `<FlightProgram>` з алгоритмом прогнозного перехоплення (`PredictIntercept`). 4. Згенерувати радар: Включати в крафт блок радара з режимом виявлення балістичних цілей.

Інтегруючи ці елементи, ми перетворюємо простий генератор крафтів на потужний інструмент для проектування складних, автономних систем, що відповідають реальним принципам роботи систем ПРО. Це виходить далеко за рамки простої генерації XML та є ключовим елементом для досягнення поставленої мети дослідження.

Процедура завантаження та використання на **Android 16+**

Однією з найбільших труднощів для гравців на нових версіях Android, зокрема на Android 16, є жорсткі обмеження на доступ до глобальної файлової системи, зокрема до каталогу `/Android/data/` . Це робить стандартний процес завантаження крафтів через меню гри або зовнішніх додатків непрактичним або неможливим. Тому для успішної реалізації інструменту необхідно надати користувачеві чіткі та надійні інструкції щодо завантаження згенерованих XML-файлів. Найнадійнішим методом є використання спеціалізованого файлового менеджера, таким як **MT Manager**.

MT Manager є популярним інструментом в гравців *SimpleRockets* завдяки своїй здатності обходити обмеження Android та надавати доступ до захищених каталогів гри, включаючи `Android/data/` [3] [17]. Процес завантаження файлу, сгенерованого нашим інструментом, використовуючи MT Manager, виглядає наступним чином:

Крок **1:** Підготовка у грі 1. Запустіть гру *Juno: New Origins*. 2. Створіть простий крафт (наприклад, лише з командного модуля) та збережіть його під будь-якою назвою, наприклад, `XML_Test` . Це створить "заглушковий" файл у каталозі збережень гри.

Крок **2:** Пошук та редагування файлу за допомогою **MT Manager** 1. Встановіть додаток MT Manager з Google Play Store [3] . 2. У MT Manager перейдіть до внутрішньої пам'яті вашого пристрою. 3. Знайдіть шлях до папки гри. Шлях може бути таким: `Internal Storage/Android/data/com.JunoNewOrigins.JNO/files/Crafts/` або `Internal Storage/JunoNewOrigins/Crafts/` . Якщо папка `Crafts` не видно, можливо, вам потрібно дозволити доступ до `Android/data` у налаштуваннях MT Manager. 4. Знайдіть там файл, який ви створили в грі (напр., `XML_Test.sr2-craft`). 5. Довго натисніть на цей файл і виберіть опцію "Відкрити як текст" або аналогічну. Це відкриє файл у вбудованому текстовому редакторі MT Manager.

Крок **3:** Заміна вмісту та збереження 1. У вікні редактора видаліть весь існуючий вміст файлу. 2. Скопіюйте весь згенерований XML-код (від `<?xml`

`version="1.0" ...` до `</Craft>`) та вставте його на його місце. 3. Перед збереженням важливо перевірити два параметри у MT Manager:

- Кодування: Переконайтеся, що вибране кодування є `**UTF-8**`. Не `UTF-8 BOM`! Вибір неправильного кодування є найпоширенішою причиною помилки "Invalid craft" [4].
- Розширення файлу: Переконайтеся, що файл зберігається з розширенням `.sr2-craft` і не має доданого `.txt`.

1. Натисніть кнопку збереження (💾).

Крок **4:** Завантаження у гру 1. Поверніться до гри *Juno: New Origins*. У меню "Крафти" -> "Мої крафти" знайдіть свій файл `XML_Test`. 2. Завантажте його. Гра може запропонувати конвертувати або оновити крафт. Підтвердьте це. 3. Якщо все пройшло успішно, ви побачите свій новий, згенерований крафт, готовий до використання.

Нижче наведена таблиця з найпоширенішими проблемами та їхніми рішеннями, які користувачеві можуть допомогти при завантаженні файлу.

Проблема	Опис	Рішення
`Invalid craft file`	Гра не може прочитати XML-файл.	Перевірте кодування файлу в MT Manager. Воно повинно бути **`UTF-8 без BOM`**. Перевірте, чи немає синтаксичних помилок у XML.
Крафт "розвалюється"	Частина крафту не є з'єднаними.	Перевірте блоки `<Connection>`. Координати та точки з'єднання (`attachPoint`) повинні бути коректними.
Файл автоматично стає `.txt`	Система опинилася, що файл не може бути відкрито у програмі.	У налаштуваннях телефону увімкніть "Показувати розширення файлів". Перейменуйте файл вручну, прибравши `.txt`.
Гра вилітає при завантаженні	Гра неспроможна обробити крафт і завершує роботу.	Спростіть XML-файл. Видаліть всі візуальні ефекти, `FlightProgram` або залиште тільки функціонал. Перевірте, чи всі лапки `"` закриті.

Надання цієї детальної інструкції є незамінним елементом дослідження, оскільки воно безпосередньо вирішує найбільшу практичну проблему користувача, пов'язану з обмеженнями платформи Android, і гарантує, що згенерований інструмент буде не просто теоретично правильним, а й реально корисним у грі.

Стратегічний синтез та рекомендації з реалізації

На підставі проведеного дослідження, створення веб-сайту для генерації робочих XML-крафтів для *Juno: New Origins* є абсолютно вірогідною та практичною метою. Ключовим висновком є те, що найефективніший підхід полягає не у використанні прямих AI-промптів для створення XML, а в розробці спеціалізованого **Python-**скрипта-генератора на базі шаблонів. Такий підхід гарантує створення технічно валідних, сумісних та фізично стабільних конструкцій, уникаючи поширених помилок, пов'язаних з некоректними координатами та з'єднаннями .

Рекомендована архітектура для реалізації проекту виглядає наступним чином: 1. Платформа**:** Використання **GitHub Codespaces** як ідеальної платформи для розгортання завдяки її повній інтеграції з Python, Flask та Git, що значно спрощує процес розробки та доставки [10] [11]. 2. Технологічний стек**: Python** для бекенд-логіки генерації XML, **Flask** для створення простого веб-інтерфейсу, та стандартних **HTML/CSS/JavaScript** для фронтенду [9]. 3. Механізм генерації**:** Створення бібліотеки на Python, яка будує XML-структуру за допомогою `xml.etree.ElementTree`. Ця бібліотека повинна використовувати перевірені шаблони для різних типів крафтів (THAAD, Patriot та ін.), вбудовуючи не лише фізичні параметри, але й логіку польоту для автономного перехоплення балістичних цілей . 4. Користувацький інтерфейс**:** Розробка максимально простого веб-інтерфейсу з однією формою для введення параметрів (тяга, довжина, кількість ступенів) та випадаючим списком для вибору типу крафту . 5. Документація та інструкції**:** Створення окремого розділу "Допомога" на сайті, який містить крок-за-крокові інструкції щодо завантаження згенерованих файлів на Android 16+ за допомогою додатку **MT Manager**. Це критично важливо для подолання обмежень файлової системи на нових версіях Android .

Реалізація цього плану дозволить створити надійний, корисний та доступний інструмент, який вирішує конкретну проблему гравців та значно полегшує процес проектування складних крафтів. Це не просто конструктор, а інтелектуальний помічник, що опирається на знання механіки гри та технічних обмежень платформи.

en

1. Where are quick save files? :: Juno: New Origins General Discussions https://steamcommunity.com/app/870200/discussions/0/1649918688127761009/
2. [XML] https://stackoverflow.com/sitemap-questions-227.xml https://stackoverflow.com/sitemap-questions-227.xml
3. MT Manager（A file manager software developed by Nanjing ... https://baike.baidu.com/en/item/MT%20Manager/652107
4. C++实现XML文件生成完整指南与项目实战原创 - CSDN博客 https://blog.csdn.net/weixin_36303807/article/details/151863206
5. [PDF] acl-ijcnlp 2015 https://aclanthology.org/W15-31.pdf
6. [XML] https://de-repo.openeuler.org/openEuler-22.03-LTS/source/repodata ... https://de-repo.openeuler.org/openEuler-22.03-LTS/source/repodata/ba07cf07d41e29ec416715646f9f58501fe1db131311132d6a76325309451209-filelists.xml.gz
7. [XML] https://mirrors.aliyun.com/almalinux-vault/8.4/devel/Source/repodata ... https://mirrors.aliyun.com/almalinux-vault/8.4/devel/Source/repodata/256760f49bfa245f27443526de9f705382bd358fbe70f83dab0942e466fcb196-filelists.xml.gz
8. Changes - curl https://curl.se/changes.html
9. [PDF] Flask Documentation (2.3.x) https://flask-doc-pdf-gen.readthedocs.io/_/downloads/en/latest/pdf/
10. How GitHub Codespaces Saved My Magento2 Project - LinkedIn https://www.linkedin.com/posts/karpovich-ivan_magento2-frontend-warden-activity-7352952733202530304-qpD-
11. What's new in Microsoft Fabric? archive https://learn.microsoft.com/en-us/fabric/fundamentals/whats-new-archive
12. Docs Xlwings | PDF | Microsoft Excel | Software Development - Scribd https://www.scribd.com/document/724922949/Docs-Xlwings
13. Device restriction settings for Android in Microsoft Intune https://learn.microsoft.com/en-us/intune/intune-service/configuration/device-restrictions-android-for-work
14. How do I access the WordPress files on Azure? - Stack Overflow https://stackoverflow.com/questions/79334148/how-do-i-access-the-wordpress-files-on-azure

15. How can I validate an email address using a regular expression? https://stackoverflow.com/questions/201323/how-to-validate-an-email-address-using-a-regular-expression/51332395
16. Understanding the difference and implications of the /MT and /MD ... https://stackoverflow.com/questions/79329235/understanding-the-difference-and-implications-of-the-mt-and-md-compiler-option
17. Thumb: A forensic automation framework leveraging MLLMs and ... https://www.sciencedirect.com/science/article/pii/S2666281725000885
18. A comprehensive review on key technologies toward smart ... https://link.springer.com/article/10.1007/s10462-025-11342-3
19. [PDF] vSRX Virtual Firewall Deployment Guide for Private and Public ... https://www.juniper.net/documentation/us/en/software/vsrx/vsrx-consolidated-deployment-guide/vsrx-consolidated-deployment-guide.pdf
20. git - Fastest way to download a GitHub repository - Stack Overflow https://stackoverflow.com/questions/6466945/fastest-way-to-download-a-github-repository
21. (PDF) Space Flight Software Systems: An approach to ... https://www.researchgate.net/publication/359602735_Space_Flight_Software_Systems_An_approach_to_understanding_their_Open_Source_Framework_Paradigm
22. GitHub codespaces CORS issues - Stack Overflow https://stackoverflow.com/questions/73022763/github-codespaces-cors-issues
23. Advanced Visual Studio Code for Python Developers https://realpython.com/advanced-visual-studio-code-python/
24. How to re-authorize the OAuth Application 'Git Credential Manager' https://stackoverflow.com/questions/66475833/how-to-re-authorize-the-oauth-application-git-credential-manager
25. Unity 5.4.0f3 https://unity.com/releases/editor/whats-new/5.4.0f3
26. How to create a file in memory for user to download, but not through ... https://stackoverflow.com/questions/3665115/how-to-create-a-file-in-memory-for-user-to-download-but-not-through-server
27. Javascript: a SIMPLE way to save a text file? - Stack Overflow https://stackoverflow.com/questions/65050679/javascript-a-simple-way-to-save-a-text-file/73775602
28. How do I create a GUID / UUID? - javascript - Stack Overflow https://stackoverflow.com/questions/105034/how-to-create-a-guid-uuid?page=3&tab=oldest
29. JavaScript-秘籍第三版-全- - 绝不原创的飞龙- 博客园 https://www.cnblogs.com/apachecn/p/19225080